



**CHANDIGARH
UNIVERSITY**

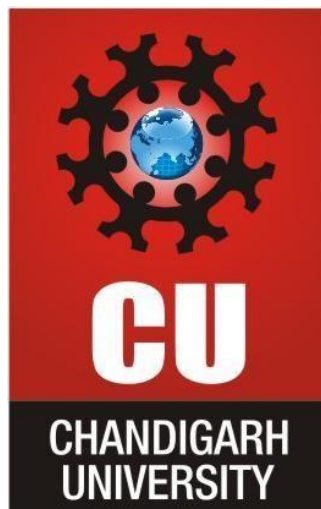
Discover. Learn. Empower.

**NAAC
GRADE A+**

Accredited University

**DEPARTMENT
OF
APEX INSTITUTE OF TECHNOLOGY
[AIT – CSE]**

CHANDIGARH UNIVERSITY



LAB MANUAL

COURSE NAME	FULL STACK - II
COURSE CODE	24CSP - 337
PROGRAM CODE	CS221

Submitted By:

Akhila (24BAI70855)
24AIT_NTPP-2[A]

Submitted To:

Ms. Lovleen Kaur (E20454)
Assistant Professor

Table of Contents

Unit	Experiment Name	Date	Marks				Signature
			LW [12]	FW [10]	W [8]	TM [30]	
1	1.1 Post Composer UI with Platform Constraints						
	1.2 Draft Management & API Integration						
	2.1 Redux Toolkit Setup & State Design						
	2.2 Optimized Selectors & Performance Handling						
	3.1 JWT-Based Authentication System						
	3.2 Role-Based Access & Route Protection						
	4.1 Interactive Calendar for Post Scheduling						
	4.2 Performance Optimization & Testing						
2	5.1 REST API Design & Implementation						
	5.2 Global Exception Handling & Logging						
	6.1 Pagination & Sorting Implementation						
	6.2 Caching & Query Optimization						
	7.1 Secure APIs with JWT & RBA						
	7.2 Encryption & Secure Credential Storage						
3	8.1 Kafka Producer-Consumer Implementation						
	8.2 Reliability: Retry, DLQ & Idempotency						
	9.1 Continuous Integration Pipeline Setup						
	9.2 Continuous Deployment & Release Strategies						
	10.1 Full-Stack System Integration						
	10.2 Capstone Project						

Experiment – 2

Name: AKHILA KONDA

UID: 24BAI70855

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance: 19/07/2026

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. **AIM:** To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE:

1. To understand the concept of global state management.
2. To implement Redux Toolkit for scalable state handling.
3. To design a normalized state structure.
4. To perform CRUD (Create, Read, Update, Delete) operations using Redux.
5. To connect React components with the Redux Store using React-Redux hooks.
6. To reduce prop drilling and improve application maintainability.

3. SOFTWARE REQUIREMENTS:

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. React-Redux
8. Redux Toolkit

4. THEORY

Redux Toolkit is the official and recommended way to manage state in React applications. It provides a **centralized store** where application data is stored and managed. Instead of passing data from one component to another using props (prop drilling), Redux allows all components to access the same data from a single source called the **Redux Store**.

Redux Toolkit simplifies Redux development by reducing boilerplate code and providing utilities such as `createSlice()` and `configureStore()`. A slice contains the application's state, reducers, and actions in one place. Reducers update the state based on dispatched actions, while React components access the state using `useSelector()` and update it using `useDispatch()`.

In this experiment, posts are stored in the Redux Store, and CRUD operations such as Add, Read, Update, and Delete are performed efficiently. This centralized approach improves scalability, maintainability, and performance of React applications.

5. IMPLEMENTATION/PROCEDURE

1. create a new React project using Create React App.
2. Install **Redux Toolkit** and **React-Redux** using npm.
3. Create a Redux Store using `configureStore()`.
4. Create a Slice (`postSlice.js`) using `createSlice()`.
5. Define the initial state containing posts.
6. Implement reducers for CRUD operations (Add and Delete posts).
7. Configure the store in `store.js`.
8. Wrap the application with the `<Provider>` component in `index.js`.
9. Use `useSelector()` to read data from the Redux Store.
10. Use `useDispatch()` to dispatch actions such as adding and deleting posts.
11. Run the application using `npm start`.
12. Verify that posts are added and deleted successfully using centralized state management.

6. CODE

Store.js

```
import { configureStore } from "@reduxjs/toolkit";
import postReducer from "../features/posts/postSlice";
```

```
export const store = configureStore({
  reducer: {
    posts: postReducer
  }
});
```

Postslice.js

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const initialState = {
  posts: []
};
```

```
const postSlice = createSlice({
  name: "posts",
  initialState,
  reducers: {
    addPost: (state, action) => {
      state.posts.push(action.payload);
    },
```

```
deletePost:(state,action)=>{
  state.posts=state.posts.filter(
    post=>post.id!==action.payload
  );
}

});

export const {addPost,deletePost}=postSlice.actions;
```

App.js

```
import { useState } from "react";
import { useDispatch,useSelector } from "react-redux";
import { addPost,deletePost } from "../features/posts/postSlice";

function App() {

  const[text,setText]=useState("");

  const posts=useSelector(state=>state.posts.posts);

  const dispatch=useDispatch();

  const handleAdd=()=>{

    if(text==="") return;

    dispatch(addPost({
      id:Date.now(),
      text:text
    }));

    setText("");

  };

  return(

    <div>

      <h2>Redux Post Manager</h2>

      <input
        value={text}
        onChange={(e)=>setText(e.target.value)}
        placeholder="Enter Post"
      />

      <button onClick={handleAdd}>Add</button>

      {posts.map(post=>

        <div key={post.id}>

          <p>{post.text}</p>
```

```
<button onClick={()=>dispatch(deletePost(post.id))}>
Delete
</button>

</div>

)}}

</div>

);
import { createSlice } from "@reduxjs/toolkit";

const initialState={
  posts:[]
};

const postSlice=createSlice({

  name:"posts",

  initialState,

  reducers:{

    addPost:(state,action)=>{
      state.posts.push(action.payload);
    },

    deletePost:(state,action)=>{
      state.posts=state.posts.filter(
        post=>post.id!==action.payload
      );
    }

  }

});

export const {addPost,deletePost}=postSlice.actions;

export default postSlice.reducer;
}

export default App;
```

7. OUTPUT

The application successfully implements centralized state management using Redux Toolkit. Users can add and delete posts through the Redux Store. The application uses a single source of truth for managing state, reducing prop drilling and improving code maintainability. Components automatically update whenever the Redux state changes.

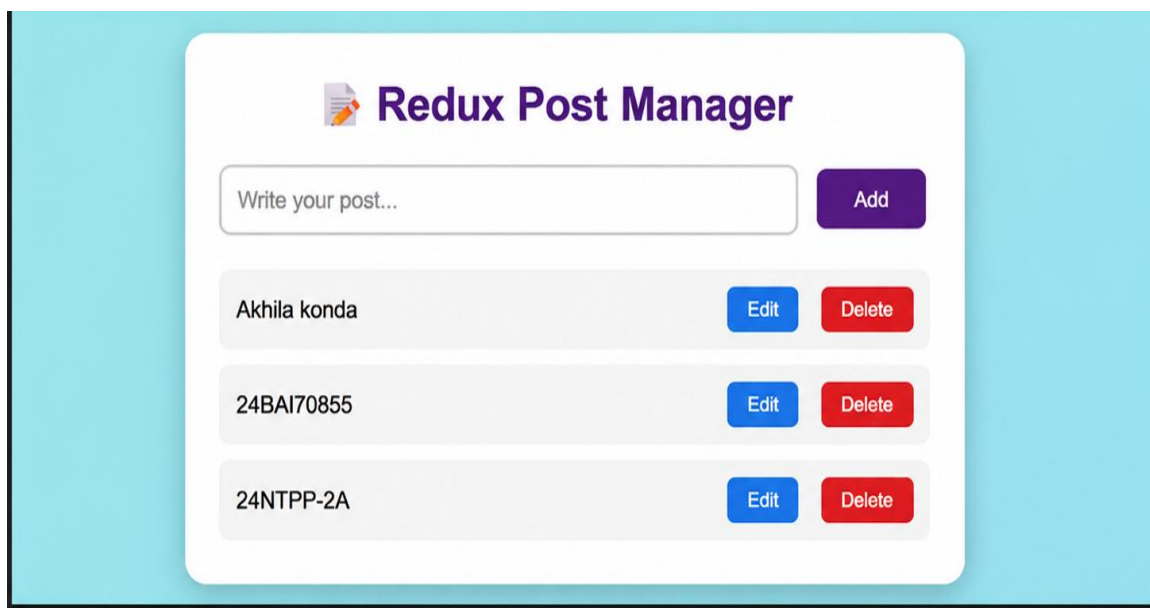


Fig 1.1: Redux Toolkit Post Management System

8. LEARNING OUTCOME

- Understand the concept of centralized state management.
- Learned how to use Redux Toolkit to simplify Redux implementation.
- Configured a Redux Store using `configureStore()`.
- Created slices and reducers using `createSlice()`.
- Performed CRUD operations using Redux actions.
- Used `useSelector()` to read data from the Redux Store.
- Used `useDispatch()` to update the Redux Store.
- Reduced prop drilling by managing global state centrally.
- Improved the scalability and maintainability of the React application.

EXPERIMENT – 2.2

1. **AIM:** To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. **OBJECTIVE:**

1. To understand the concept of derived state.
2. To implement memoized selectors using **Reselect**.
3. To reduce unnecessary component re-renders.
4. To improve performance in large-scale React applications.
5. To efficiently retrieve data from the Redux Store using selectors.

3. **SOFTWARE REQUIREMENTS:**

1. Windows/Linux/macOS
2. Node.js (LTS Version)
3. NPM (Node Package Manager)
4. Visual Studio Code
5. Google Chrome / Microsoft Edge
6. React.js
7. React-Redux
8. Command Prompt / Terminal

4. **THEORY**

As React applications become larger, managing and accessing state efficiently becomes an important challenge. Every time the Redux store updates, selectors may recompute values even when the required data has not changed. These unnecessary computations can reduce application performance.

Redux Toolkit provides a simple way to manage centralized application state. To optimize state access, Redux applications use **selectors**, which are functions that retrieve specific data from the Redux Store.

A **memoized selector** stores the previously computed result and recalculates it only when its input changes. This process is called **memoization**. The **Reselect** library provides the `createSelector()` function, which creates memoized selectors. By avoiding repeated calculations, memoized selectors improve application speed and reduce unnecessary rendering.

Another important concept is **derived state**. Derived state is data that is calculated from existing state instead of being stored separately. For example, the total number of posts or filtered posts can be calculated whenever required instead of maintaining another state variable.

React also provides optimization techniques such as **React.memo()**, which prevents a component from re-rendering when its props have not changed, and **useMemo()**, which caches expensive calculations between renders.

Using memoized selectors together with React optimization techniques results in faster rendering, reduced CPU usage, better scalability, and improved maintainability of large React applications.

5. IMPLEMENTATION/PROCEDURE

1. Create a new React application using Create React App.
2. Install Redux Toolkit, React-Redux, and Reselect.
3. Create a Redux Store using `configureStore()`.
4. Create a slice containing the application state.
5. Define basic selectors to access the Redux Store.
6. Create memoized selectors using `createSelector()`.
7. Compute derived state such as total posts and filtered posts.
8. Connect React components to the Redux Store using `useSelector()` and `useDispatch()`.
9. Use `React.memo()` and `useMemo()` wherever necessary to reduce unnecessary rendering.
10. Run the application using `npm start`.
11. Verify that selectors recompute values only when the state changes.

6. CODE

Store.js

```
import { configureStore } from "@reduxjs/toolkit";  
import postReducer from "../features/posts/postSlice";
```

```
export const store = configureStore({  
  reducer: {  
    posts: postReducer,  
  },  
});
```

PostSlice.js

```
import { createSlice } from  
"@reduxjs/toolkit";
```

```
const initialState = {  
  posts: [  
    { id: 1, text: "React Learning" },  
    { id: 2, text: "Redux Toolkit" },  
    { id: 3, text: "Memoized Selector" }  
  ]  
};
```

```
const postSlice = createSlice({  
  name: "posts",  
  initialState,
```

```
reducers: {

  addPost: (state, action) => {
    state.posts.push(action.payload);
  },

  deletePost: (state, action) => {
    state.posts = state.posts.filter(
      post => post.id !==
action.payload
    );
  }

}

});

export const { addPost, deletePost } =
postSlice.actions;

export default postSlice.reducer;
```

PostSelector.js

```
import { createSelector } from
"reselect";

const selectPosts = (state) =>
state.posts.posts;

export const selectAllPosts =
createSelector(
  [selectPosts],
  (posts) => posts
);

export const selectPostCount =
createSelector(
  [selectPosts],
  (posts) => posts.length
);
```

App.js

```
import { useState } from "react";
import { useDispatch, useSelector } from "react-redux";
import { addPost, deletePost } from "../features/posts/postSlice";
import {
  selectAllPosts,
  selectPostCount,
} from "../selectors/postSelectors";

function App() {

  const [text, setText] = useState("");

  const posts = useSelector(selectAllPosts);

  const totalPosts = useSelector(selectPostCount);
  const dispatch = useDispatch();
  const handleAdd = () => {

    if (text === "") return;

    dispatch(
      addPost({
        id: Date.now(),
        text,
      })
    );
    setText("");
  };

  <div key={post.id}>

    <p>{post.text}</p>

    <button
      onClick={() => dispatch(deletePost(post.id))}
    >
      Delete
    </button>

  </div>

  )}

</div>

);
}
export default App;
```

7. OUTPUT

The application successfully implements memoized selectors using Reselect. Data is accessed efficiently from the Redux Store through selectors. The total number of posts is calculated as derived state without storing duplicate data. Memoized selectors prevent unnecessary recomputation, reducing component re-rendering and improving overall application performance. The application demonstrates optimized state management suitable for scalable React applications

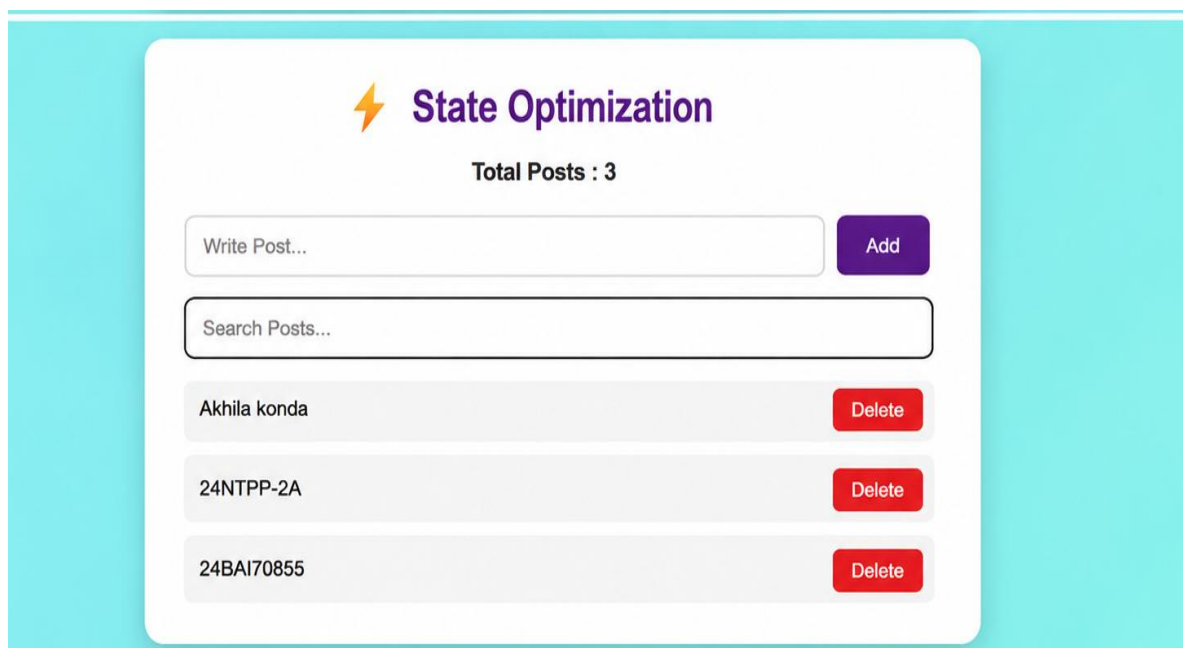


Fig 1.1: React-Redux Application with Memoized Selectors

8. LEARNING OUTCOME

- Understood the concept of centralized state access using selectors.
- Learned the purpose of memoization in React applications.
- Implemented memoized selectors using createSelector().
- Computed derived state without duplicating data.
- Reduced unnecessary selector recomputations.
- Improved application rendering performance.
- Understood the use of React.memo() and useMemo().
- Connected React components with optimized Redux selectors.